

About Donna

What is Donna?

Donna is a friendly, focused frontend built on top of **LQ-AI**, an open-source legal operating system. Where LQ-AI exposes the full power of a legal AI platform, Donna presents a streamlined experience designed for day-to-day legal work — clean, approachable, and tuned for the tasks that matter most.

The **Powered by LQ-AI** callout at the top of every page in this guide links to a deeper explanation of the backend that drives Donna's answers, document intelligence, and knowledge retrieval.

A tour of the sidebar

Everything in Donna is reachable from the left sidebar. Here is what each entry does:

- **Assistant** — the main chat interface; start a conversation, attach files, and apply skills or prompts from your library.
- **Projects** — organise your work by matter; chats and documents can be scoped to a project so context stays together.
- **Workflows** — skills, playbooks, saved prompts, and automations that run or accelerate recurring tasks.
- **Tabular** — structured document review; extract and compare fields across a set of files in a grid.
- **About** — this guide.
- **Settings** — account, model routing, and provider configuration.

How this guide is organised

The rail on the left of this guide has one page per area of Donna. Start with the **Assistant** page — it covers the core chat loop that underlies everything else. From there, work through **Projects**, **Workflows**, **Automations**, and **Tabular** in any order that suits your workflow. The **Knowledge**, **Models**, and **Trust & citations** pages cover the infrastructure behind Donna's answers.

A standing reminder

Donna's answers are **not legal advice**. They are AI-generated responses that can contain errors or omissions. Always apply your own professional judgment, and verify any information before relying on it in a legal matter.

The Assistant

Starting a conversation

The **Assistant** is Donna's core interface. When you open the app you land on a central composer — a single input field that greets you by name. Type your question or task there and press **Enter** (or the send arrow) to start a new chat. Donna streams the answer back in real time and the conversation moves into its own chat page, where you can continue sending follow-up messages using the composer at the bottom of the screen.

Every chat lives at its own URL, so you can bookmark it, share the link internally, or return to it later from the sidebar's chat history. Within a chat the full message thread is preserved and the composer stays ready at the bottom for follow-up turns.

◆ Enhance — rewriting a vague draft

The **Enhance** button (the sparkle icon labelled **Enhance** in the composer toolbar) takes what you have typed so far and rewrites it into a fuller, more precise prompt before you send it. It is particularly useful when you have a rough idea but are not sure how to phrase it for best results.

The flow is three steps:

- **Run** — click **Enhance**. The button briefly shows "Enhancing..." with a cancel option while the backend rewrites your draft.
- **Preview** — a preview card appears above the text box showing the expanded prompt and, optionally, a collapsible "Why these changes" list explaining each edit.
- **Accept or discard** — click **Use this** to replace the text in the composer with the enhanced version, or **Discard** to go back to your original draft. Neither action sends the message; you can still edit before hitting send.

If the backend finds nothing to improve it shows a brief "No changes suggested" note and the composer is unchanged. Enhance is context-aware: if you have skills attached to the message, those skill names are sent along so the rewrite can take your chosen workflow into account.

Attaching skills

Skills are structured workflows that guide Donna's response — for example, a contract review checklist or a clause extraction routine. To attach one, click the **Skill** button (a + icon) in the

composer toolbar. A small search panel opens; type a keyword to find skills by name or description and click the skill you want to add.

Attached skills appear as pill badges inside the composer. Some skills expose required or optional input fields — those appear inline between the pills and the text area so you can fill them in before sending. Required fields must be filled before the send button becomes active. Remove a skill at any time by clicking the × on its pill. You can attach more than one skill to a single message; each is sent alongside your prompt.

Attaching files

To ground an answer in a specific document, click the **paperclip** button in the toolbar or drag a file directly onto the composer. You can attach up to **16 files per message**. Each file appears as a pill showing the filename and its upload status ("uploading" → "pending" → "processing" → "ready", or "failed"). The send button stays disabled until every attached file has finished processing — a failed file must be removed before you can send. Remove any file at any time with the × on its pill.

Files attached to a message are sent to the backend alongside your text and are available to the model for that turn. If you need to make a document available across many chats, upload it to a project's knowledge base instead — see the **Knowledge** page of this guide for details.

Saved prompts

The **Prompts** button (bookmark icon) opens your saved prompt library. Click a saved prompt to insert its text at the cursor position in the composer — it splices in without overwriting what you have already typed. You can also save the current draft directly from the panel: click + **Save current draft as a prompt**, give it a name and optional tags, and save it for future reuse.

Choosing a model

The model picker in the composer toolbar shows the currently selected alias (for example **smart**) alongside the resolved model it maps to (for example "Opus 4.7"). Click it to open a menu grouped into **Cloud** and **Local** sections. Choose any model for the next message; your selection persists across turns in the same browser session.

When a chat is scoped to a project that requires a minimum capability tier, lower-tier models appear greyed out in the picker and cannot be selected. Donna automatically promotes your chosen model to the floor tier in that case so the answer meets the project's quality requirements. The **Models** page

of this guide explains tiers, cloud vs. local routing, and how to configure model preferences in Settings.

Reading the answer: citations

When Donna's answer draws on documents in a knowledge base or on files you attached, it embeds numbered citation pills inline in the text — small superscript markers like ¹ or ². Hover a pill to see a popover with the source text excerpt, the document filename, and the page number if available. The popover header is colour-coded: green for verified citations, amber for those with caveats, and red for citations that could not be matched to a source.

Click a citation pill (or press Enter/Space when it is focused) to open the **Document panel** on the right side of the chat, which shows the full source document with the cited passage highlighted. This lets you read the answer and its source side by side without leaving the chat. The **Trust & citations** page of this guide explains how citations are verified and what each colour state means.

Receipts drawer

Every chat exposes a **Receipts** button in the top-right corner of the chat header. Click it to open the receipts drawer, which lists every backend event that occurred during the conversation — retrieval steps, skill invocations, model calls, and more — in chronological order. Use the kind-filter chips at the top of the drawer to show or hide event categories. You can also export the full event log as a JSONL file via the **Export** link in the drawer header, useful for debugging or auditing a response.

Projects

What is a Project?

A **Project** groups related chats, files, knowledge bases, skills, and standing context under one name. Instead of starting every conversation from scratch, you give Donna a shared foundation — background notes, uploaded documents, and relevant knowledge — that applies automatically to every chat inside that Project.

A note on naming: the sidebar entry is labelled **Projects**. In some parts of the interface (page headings, form labels, archive dialogs) you will see the word **matter** instead. These refer to the same thing; the terms are interchangeable.

Creating a Project

Open **Projects** in the sidebar. Click **New matter** in the top-right corner. A dialog asks for:

- **Matter name** — required. Choose something that reflects the engagement or topic.
- **Description** — optional free text shown beneath the name in the list.
- **Privileged matter** — see below.
- **Minimum model tier** — see below.

Once created, the Project appears in the list sorted by last activity. You can **Rename** it at any time from the Project detail page; you can also **Archive** it to remove it from your active list without deleting any chats inside it.

Privilege and minimum model tier

Checking **Privileged matter** does two things:

- Every chat started inside the Project is flagged as privileged in the audit log.
- A **Minimum model tier** (1–5) becomes required — you must choose one before you can save a privileged Project. Higher tiers require cloud-hosted models.

Privileged Projects display a **Privileged** badge — a small lock-icon pill — next to the Project name in the list and on the Project detail page. Non-privileged Projects can optionally set a minimum tier without enabling the privileged flag.

Attaching files

The **Files** section on a Project's detail page lets you attach documents directly to the Project. When no files have been added yet, a drop zone is shown — drag files onto it or click to open the file picker. After the first upload an + **Add file** button appears below the file list so you can keep adding more. Uploaded files are available to chats in that Project and to any linked knowledge base workflows.

Knowledge bases and skills

A Project can be linked to one or more **Knowledge** bases. Linking a knowledge base makes its indexed documents retrievable during every chat in the Project. Use the **Knowledge** section on the detail page to link or unlink bases; the **Manage** link takes you to a knowledge base's own settings.

The **Skills** section lets you attach named skills to a Project. Attached skills appear as removable tags and are available to Donna when answering chats scoped to that Project.

Standing context

The **Context** section holds free-form Markdown that Donna receives at the start of every chat in the Project — no matter who starts it or when. Use it for background facts, formatting preferences, recurring instructions, or any information that would otherwise need to be repeated in every prompt. The field supports up to 100 KiB of text; a live byte counter is shown as you type.

Scoping a chat to a Project

From the Project detail page, click + **New chat in this matter**. The chat opens already scoped to the Project: Donna receives the standing context, can draw on linked knowledge and skills, and sees the attached files. The chat list at the bottom of the detail page shows all chats ever started in that Project, with a message count for each.

If a Project is marked privileged, the minimum tier setting is also enforced for those chats, ensuring they are handled by a model at or above the required capability level.

Workflows

The **Workflows** section is the single place in Donna where you save and organise reusable building blocks. One sidebar entry opens a hub with four tabs — **Skills**, **Playbooks**, **Prompts**, and **Automations** — each covering a different kind of reuse.

Skills

A skill is a saved block of instructions that shapes how Donna responds to a message. When you attach a skill to the composer, those instructions are automatically included with your next send — you do not need to retype them each time.

The **Skills** page lists your own skills under **Your skills** and a searchable catalogue of built-in skills below. Built-in skills cover common legal tasks and cannot be edited directly, but you can **Fork** any of them to create your own copy — the fork appears in **Your skills** and you can edit it freely.

To author a skill from scratch, press **New skill**. The edit form lets you set a name, description, version, an optional slash command alias (e.g. **/nda**), tags, and the skill body — the instructions Donna receives. You can archive a skill you no longer need; it will be removed from the composer.

Some skills declare **inputs** — structured fields the skill expects you to fill in before sending. When you attach such a skill in the composer, an inputs panel appears inline. Required inputs are flagged;

a skill with unfilled required inputs cannot be sent until they are provided. Optional inputs can be expanded separately. Input types include text, integer, boolean toggle, and enum (drop-down).

Playbooks

A playbook is a set of negotiation positions Donna applies to a contract document. Each position describes a clause topic and the stance Donna should take — preferred language, fallback, and non-negotiable limits — so you get consistent redlines every time instead of rewriting guidance from scratch.

The **Playbooks** page groups available playbooks by contract family. Tap any playbook to open its detail view, which shows the full list of positions in order. From the detail view you can **Apply to a document** (admin accounts only in this version), **Edit** the playbook if you own it, **Duplicate** it to create a private copy, or **Delete** it.

To create a new playbook, press **New playbook** from the list. Two paths are available:

- **Generate from documents** — pick existing matter files and name a contract type; Donna drafts a full playbook from those documents and shows a review editor before you save.
- **Start from scratch** — open the playbook editor directly and build positions by hand.

Saved Prompts

Saved prompts are reusable text snippets accessible from the chat composer. When you find yourself typing the same phrasing repeatedly — a standard instruction, a preferred opener, a recurring question — you can save it once and insert it with two clicks.

The **Prompts** page lists all your saved prompts. Press **New prompt** to create one with a name, body text, and optional tags. You can edit or delete any prompt from the same list.

In the composer, the **Prompts** button opens a small picker above the input area. Type to filter by name or tag, then click a result to insert its text at your cursor. You can also save whatever you have already typed as a new prompt directly from the picker — name it and press **Save** without leaving the chat.

Automations

The **Automations** tab is where Donna runs work on its own — one-off runs, cron-style schedules, and watches that fire when new documents arrive in a knowledge base, each with a full receipt of what the agent did and what it produced. See the dedicated [Automations guide](#).

Automations

Automations let Donna run work on its own — a playbook or a skill executed against a knowledge base in the background, on demand or on a schedule. Every run leaves a full transparency receipt: what the agent did, what it cost, why it stopped, and what it produced.

Turning automations on

Automations are off by default. Enable them with the **Automations** toggle in **Settings** → **Preferences**. Until then, automation pages show an inline **Automations are off** notice with an enable button, so you can opt in right where you need it.

Where to find it

Automations is the fourth tab of the **Workflows** hub, alongside Skills, Playbooks, and Prompts. Inside it, five views cover the lifecycle: **Sessions** (every run, past and live), **Schedules**, **Watches**, **Notifications**, and **Review**.

Starting a run now

Press **Run now** to start a one-off session. Choose what to run — a **playbook** or a **skill** — then pick the **target knowledge base** the run works against (required), optionally link a **matter**, and optionally set a **cost cap** in USD. Donna starts immediately and takes you to the live receipt.

Schedules and watches

- **Schedules** run the same configuration on a recurring timer. You describe the cadence with a cron expression — the form previews it in plain language before you save.
- **Watches** trigger a run automatically when new documents arrive in a knowledge base — useful for reviewing incoming material without checking manually.

Both are editable after creation — including reassigning or unlinking the matter — and both accept a per-run cost cap.

Receipts: what the agent did

Open any session to see its receipt. The header shows the run's status, how it was triggered, total cost, and — for finished runs — why it stopped. The **Activity** timeline lists every phase the run moved through and every tool call it made, in order. While a run is live the page updates automatically every few seconds.

Results: what the run produced

The **Results** section shows the run's work product: its findings, in the order the run produced them, each with a severity badge and a severity summary up top. Runs that opt in to **Save run documents** can also produce document-grade results — memos saved to the run's target knowledge base, listed under **Documents** on the receipt, where you can open them inline or download them. A run can also propose **memories** — durable notes it suggests keeping — listed under **Memories this run proposed**. Proposed memories can be kept or dismissed right on the receipt, or managed in the **Review** view. The Review view also lists **precedents** — recurring patterns the agent noticed — which you can dismiss or promote into a matter's context (you approve the final write). Results stream in live while the run works.

Notifications

When a run finishes, it posts to the **Notifications** view — title, summary, and a link straight to the session's receipt. Unread notifications carry a dot; **Mark read** clears them one at a time.

Cost caps

Every way of starting a run — run now, schedules, watches — accepts an optional cost cap in USD. A run that reaches its cap stops, and the receipt records that as the terminal reason. Combined with receipts and notifications, you always know what ran, what it cost, and what it produced.

Tabular review

What it is

Tabular review runs the same set of questions across many documents at once and returns the answers in a cited grid — one row per document, one column per question. Use it when you need to compare how different agreements, filings, or contracts answer the same question, or when you want to extract structured facts from a large document set without reading each file individually.

Each cell in the grid carries a confidence signal (**high**, **medium**, or **low**) and links to the source passage in the original document. You can export the finished grid to Excel or CSV for use outside Donna.

Starting a new review

Open **Tabular** in the sidebar, then click **New review**. The builder page is split into two panels: **Documents** on the left and **Columns** on the right.

To add documents, switch between the **From a matter** and **Upload** tabs. When using a matter, pick the matter from the dropdown and check each file you want to include. When uploading, drag files onto the drop zone; they are ingested immediately and added to the selection as they finish processing. Selected documents appear as removable chips below the picker.

Defining columns

The **Columns** panel offers two modes, toggled with the **Define columns / Use a table skill** control:

- **Define columns** — build your own grid ad hoc. Each column has a **Column name** (the heading that appears in the grid) and a **Column question** (what Donna should extract, e.g. "Which state's law governs?"). Use the arrow buttons to reorder columns and the × button to remove one. Click **Add column** to add more. Column names must be unique.
- **Use a table skill** — pick a pre-built table skill from the list. The skill defines the columns; you cannot edit them here. The exact column count and estimated cost appear in the cost preview before you commit.

When defining columns manually, each column also has an optional **Min. model tier** setting (tiers 1–5, or none). Setting a minimum tier tells Donna to use at least that capability level when answering that column's question. Leave it at **None** to let the system choose automatically. Higher tiers may increase the estimated cost.

Each manually-defined column also has an **Ensemble verification** checkbox. When enabled, the answers in that column are independently cross-checked before being accepted, and when you open one of a verified cell's citations, the source view shows a green ✓ **Verified** chip. Ensemble-verified cells cost more; the cost preview shows the count of ensemble-verified cells and the premium as a separate line.

Running the review

The status bar at the bottom of the builder shows the current scope — for example **4 docs × 3 cols = 12 cells** — so you can gauge the size of the run before committing. When you are ready, click **Preview cost**.

A cost preview modal appears, showing the total cell count and an estimated cost in USD broken down by model tier. The estimate is approximate. Click **Run review** to start, or **Cancel** (or press Escape) to go back and adjust the setup.

Once started, the review runs in the background. The grid page opens automatically and shows a live progress indicator while cells are being processed. If a run takes longer than expected, a note appears suggesting you reload to check again. You can click **Cancel** on the running review page to stop it early.

Reading the grid

When the review completes, the grid shows one row per document and one column per question. Each cell displays a coloured dot indicating confidence — green for high, amber for medium, red for low, and grey for a failed cell — followed by a short excerpt of the answer. A number on the right edge of a cell indicates how many source citations back the answer.

A summary line above the grid shows the total cell count and how many cells failed. If any cell shows **(failed)**, the cell detail panel (see below) will display the error reason when available.

Navigating from a cell to its source

Click any cell to open the **cell detail panel** — a slide-in drawer on the right side of the screen. The panel shows the full answer text, the confidence level, and a list of citations. Each citation is a button labelled **Source · p.N**, with a short preview of the supporting passage below it.

Clicking a citation opens the **document panel** — the same viewer used elsewhere in Donna — which loads the source document and jumps to the cited page and passage, with a green **✓ Verified** chip when the cell was ensemble-verified. Press Escape or click the × button to close the cell detail panel.

Exporting results

On any completed review, click the **Export ▼** button at the top right of the grid and choose **Excel (.xlsx)** or **CSV (.csv)**. The file downloads immediately.

History and resuming past reviews

All past and in-progress reviews are listed on the **Tabular** index page. Each row shows the run's status (completed, running, pending, failed, or cancelled), the number of documents and columns, an estimated cost if available, and the date the review was created. Click any row to open that review's grid — completed reviews show their full results immediately, while still-running reviews continue polling for updates in the background.

Knowledge bases

What is a knowledge base?

A **Knowledge base** is a named collection of documents that Donna can search when answering your questions. Instead of relying only on what the AI model already knows, Donna retrieves relevant passages from your uploaded documents and uses them to ground its answers — a technique called retrieval-augmented generation (RAG). This means answers can draw on your specific contracts, policies, research, or other materials rather than general training data.

Creating a knowledge base

Knowledge bases live inside **Projects**. Open a Project and find its **Knowledge** section: the picker there lets you link an existing KB or create a new one. When creating, you give the KB a name — pick something descriptive, such as "Acme Master Agreements" or "Company Policies Q4". Linking a KB to a Project makes its documents available to every chat in that Project.

Click a linked KB (or its **Manage** link) to open the knowledge base's own page. From there you can add files, review ingestion status, rename it, and tune its retrieval settings.

Renaming and describing a knowledge base

Open a KB and click **Rename** in the header. A dialog lets you update both the **Name** and an optional **Description**. Click **Save** to apply the changes. Press **Cancel** or hit Escape to close without saving.

Uploading documents

Inside a KB, the **Files** section shows all attached documents. If the KB is empty you will see a drop zone — drag files onto it or click to open a file picker. Once files are present, use the + **Add file** button to upload more.

After upload, each file moves through ingestion states: **Pending** → **Processing** → **Ready**. Donna polls automatically and attaches the file once it is ready. If a file is still processing after several minutes, a **Refresh** prompt appears. Files that cannot be ingested show a **Failed** status with an error message.

Recommended format: PDF. PDF files ingest reliably. Some other file types — including plain `.txt` files — may be rejected with an "unsupported type" error. If a file fails to ingest, try converting it to PDF before uploading.

To remove a document, click the × icon next to the file row. This detaches the file from the KB; it will no longer be searched when Donna retrieves context. Ready files also show a **Download** link so you can retrieve the original document directly from the KB.

How retrieval works

When you chat within a Project that has a linked knowledge base, Donna searches those documents for the passages most relevant to your query. Matching chunks are included in the context sent to the AI model, so the answer is grounded in your actual materials rather than model knowledge alone.

Each KB also exposes a **Hybrid alpha** control (in the **Advanced** section of the KB detail view). This slider blends two retrieval strategies: pure vector similarity search at one end, and full-text search (FTS) at the other. The default balance works well for most document types; you can tune it if you find one strategy returns better results for your content.

Answers that draw from a knowledge base cite their sources so you can verify exactly which document and passage the answer came from. See the **Trust & citations** page for a full explanation of how citations work in Donna.

Archiving a knowledge base

When a knowledge base is no longer needed, open it and click **Archive** in the header. Archived KBs are removed from the active list and will not be searched. This is useful for keeping the Knowledge view tidy without permanently deleting documents.

Models

Choosing a model per message

Every message you send uses a model alias — a named routing slot that the backend resolves to a concrete model at request time. You choose your alias in the composer's model picker before sending. The picker shows only chat-capable aliases; embedding and other non-chat model types are filtered out automatically.

The two most common aliases you will see are:

- **smart** — routes to the highest-capability cloud model currently configured. Choose this when quality and depth matter more than speed.
- **fast** (or other lower-tier aliases) — routes to a faster, lighter model. Useful for quick drafts, summaries, or when you want lower latency.

The picker also surfaces any installed local models (served via Ollama). Local aliases are labelled **Local** in the picker. They run on-device and never send data to a cloud provider, making them a good choice for sensitive or offline work.

Some matters enforce a minimum inference tier. When you open a matter that requires a higher-tier model than your current selection, Donna automatically promotes your choice to a model that satisfies the floor — preferring **smart** when it qualifies, otherwise the highest-tier option available — so you never accidentally send a message with an underpowered model.

Settings → Models

Workspace-level model routing is managed in **Settings** → **Models**, reachable from the sidebar. The page has three sections.

Inference categories

The **Inference categories** section lists every named alias in the system. Each row shows the category name, whether it is backed by a cloud or local model, and its current backing model label.

- **Admins** see a drop-down on every row and can reassign a category to any available cloud or local model. Changes take effect immediately — the row shows **Saving...** while the request is in flight and **Saved** on success.

- **Non-admins** see the current backing model for each category but cannot change it. A note at the bottom of the section explains that changing model routing requires an admin account.

Installed local models

The **Installed local models** card lists every Ollama model detected on your system, showing the model's label and its full identifier. If no local models are found, the card explains how to install Ollama and pull a model with `ollama pull <model>`. Adding a new model via Ollama and reloading the page is all that is needed for it to appear here and become selectable in the composer.

Provider keys

The **Provider keys** card lists each model provider the deployment knows about (Anthropic, OpenAI, and any others) with its key status — **✓ Configured**, with the key's source and last four characters, or no key yet. Keys are write-only: Donna never shows a stored key, only its last four characters.

- **Admins** paste a key into the masked input on a row and press **Add key** (or **Replace key**). Changes apply immediately — no restart needed.
- Keys set by the deployment's **environment** can be taken over by saving a runtime key on the same row. Runtime keys can be revoked with a two-step confirm; environment-managed keys cannot be revoked from the UI.
- **Non-admins** see a note that provider keys are managed by your administrator.

Trust & citations

Citation pills

When Donna's answer is backed by documents in your knowledge base, numbered **citation pills** appear inline in the response text — one pill per supporting passage. Hovering or focusing a pill opens a small popover that shows the exact quoted text from the source, the filename, and the page number when the backend provides it.

The popover header is colour-coded to signal how confidently the passage was matched:

- **Green — Verified.** The passage was found in the source as an exact or normalised match, or confirmed by ensemble verification (multiple judges agreed it is supported).

- **Amber — Caveats.** The source partially supports the claim, or the match was made by a single paraphrase judge rather than a direct text search.
- **Red — Unverified.** The citation could not be confirmed against the source document.

Clicking a pill (or pressing **Enter** or **Space** when focused) opens the **Document panel** on the right side of the screen so you can read the full surrounding context in the source.

Receipts

Every chat page has a **Receipts** button in the top-right corner. Opening it slides in a panel that lists every backend event recorded during the conversation — from your initial message, through any knowledge-base retrieval and model inference steps, to the final answer.

Each row is labelled by kind: **message**, **retrieval**, **inference**, **skill**, **audit**, or **error**. Retrieval rows show how many chunks were pulled and from how many knowledge bases. Inference rows show the model name, provider, token counts, and latency. Selecting **Details** on any row expands the raw event payload for deeper inspection.

You can filter the list by kind using the chips at the top of the drawer, and export the full event log as a JSONL file with the **Export** link in the drawer header.

Anonymization indicator

When a cloud model is used, your prompt passes through an **anonymization layer** before it leaves your environment — identifiers are stripped at the edge. Donna surfaces this as an **Anonymized** badge (with a shield-check icon) that appears beside the answer whenever the backend confirms that anonymization was applied to that inference call.

The same signal appears inside the Receipts drawer: each inference row shows either **Anonymized** (green) or **No anonymization** (grey) depending on what the backend recorded for that event. Local models, which run entirely on-device, do not send data outbound at all and therefore do not carry this indicator.

Standing disclaimer

Every chat page and the landing composer both display the notice: "**AI can make mistakes. Answers are not legal advice.**" This reflects a core design principle of Donna: the system surfaces sources and verification signals so you can judge the answer yourself — it does not replace

the professional judgement of a qualified lawyer. Always verify critical information against the underlying documents before acting on it.

Powered by LQ-AI

Donna is powered by LQ-AI, an open source legal operating system. Donna uses some, but not all, of the functionality available in LQ-AI. The interactive surfaces below walk through how LQ-AI (and Donna) work, from a request to a verified, cited answer.

Build & learn with LQ-AI

Donna is one application built on LQ-AI. Because the whole stack is open source, here is some of what else you could build — and teach — with it.

Power new applications

The same backend that powers Donna's chat, playbooks, tabular review, autonomous runs, and knowledge retrieval can front entirely different products. A clause-library browser could let transactional lawyers search, compare, and tag contract clauses against a RAG-indexed precedent bank, with the citation engine flagging each match as verified or caveated. A deposition-prep assistant could ingest case documents into a knowledge base and surface witness-specific question sets via playbooks. A compliance-checklist tool could run a saved-prompt workflow over uploaded policies, returning a structured tabular review that maps each regulatory requirement to the relevant section — or flags gaps as unverified.

Extend LQ-AI itself

LQ-AI is designed to grow through contributions. You can author new skills — focused, reusable prompt units — and chain them into playbooks that encode multi-step legal reasoning workflows. Adding a new model provider (another cloud API or a locally-hosted Ollama model) requires only a small adapter alongside the existing provider registry. The citation engine and anonymization layer are independent modules: improving redaction heuristics, expanding jurisdiction-specific citation

formats, or tuning verified/caveated/unverified thresholds are all self-contained contributions. For more ambitious work, the autonomous primitives — the planner and tool-call scaffolding that underpin multi-step tasks — are open for extension. The project's contributor guide covers the mechanics.

Learn & teach

For **law students**: the interactive playgrounds on this page let you see exactly how legal AI works under the hood — how retrieval pulls documents from a knowledge base, how the citation engine decides what counts as verified versus caveated, how anonymization strips identifying information before a prompt leaves your network. From there, building a skill or playbook for a contract type you study (NDA, SaaS agreement, merger agreement) makes a concrete course project that produces something real. The tier/refusal governance system — which controls what queries the model will and won't answer — is a live, inspectable example of AI-governance design, useful for any seminar touching on responsible AI.

For **law professors**: the open codebase and the playgrounds on this page can serve as primary teaching material for a legal-AI or law-and-technology course — students read the source rather than a vendor white paper. A "build a playbook" assignment gives students hands-on exposure to prompt engineering and workflow design in a legal context without requiring any ML background. For clinics, the architecture is thin enough that a student team can fork a Donna-style frontend, adapt it to a specific legal-aid workflow, and deploy it against the same backend in a semester — producing a tool a real organization can use.

Access to justice

Because LQ-AI is open source and supports local model providers through Ollama, the stack can be adapted for pro-bono and access-to-justice settings where commercial SaaS licensing is out of reach. A legal-aid organization can run the entire system on local hardware — no data leaves the premises and no per-query cost accrues — while still benefiting from knowledge-base retrieval, anonymization, and the citation engine. A slim Donna-style frontend scoped to housing, benefits, or

immigration matters could be deployed for self-represented litigants or intake staff with minimal ongoing cost.

How to build on LQ-AI → — the contribution path, skill authoring (with an interactive SKILL.md explorer), and the roadmap.

Explore the project: [LegalQuants / lq-ai on GitHub](#).

1. The big picture: System Architecture

LQ-AI is three services: the FastAPI backend (api/), the Inference Gateway (gateway/), and the SvelteKit web frontend (web/). They communicate over HTTP using OpenAPI-defined contracts; no service shares in-process code with another. The Gateway is the security boundary — the only component that holds provider API keys and makes outbound inference calls. This map shows the

service topology, the network boundaries, and the trust model.



[Open full-screen](#) / [Source: docs/architecture.md](#)

2. A request, end to end: Lifecycle of a chat send

When you press Enter in the composer, the message travels through at least six distinct processing stages before the model sees it — and through three more before the response reaches your screen. Each stage can add context (skill prompt, KB chunks, tier metadata), apply a policy check, or produce an audit record. This playground walks the full path so you can see where each transformation happens and which file implements it.

LQ.AI — Request Lifecycle Step through what happens server-side when a chat message is sent.

[↔ Learn](#) [View source ↗](#)

SCENARIO

Simple message

With slash-invoked skill

With knowledge base attached

Refused at tier floor

Cached / no LLM call

PLAYBACK

< ▶ > ↺

1 / 20

●

STEPS

1 User clicks Send

2 Optimistic message added to UI

3 POST /api/v1/chats/{id}/messages

4 API validates auth and chat ownership

5 API writes user message to DB

6 API writes audit log: chat.message_sent

7 API: skill resolution

8 API assembles inference request

9 POST to Gateway /chat/completions

1

#1 WEB User clicks Send

Composer captures the typed message and fires a send action.

web/src/lib/lq-ai/components/ChatPanel.svelte

2

#2 WEB Optimistic message added to UI

Message appears immediately with a "pending" state.

web/src/lib/lq-ai/components/ChatPanel.svelte

3

#3 WEB POST /api/v1/chats/{id}/messages

HTTP request sent from Web to API with content, role, attached_skills, attached_files.

web/src/lib/lq-ai/components/ChatPanel.svelte

POST /api/v1/chats/{chat_id}/messages

4

#4 API API validates auth and chat ownership

JWT verified; chat confirmed as belonging to this user (404 on cross-user).

api/app/api/chats.py

fn: send_message()

5

#5 DB API writes user message to DB

chat_messages row inserted: role=user, applied_skills, user_message_id.

api/app/api/chats.py

DB write: chat_messages fn: send_message()

Verify in source

Description

Copy

Step: 1 of 20 — User clicks Send

Component: Web

File: web/src/lib/lq-ai/components/ChatPanel.svelte

ALL FILES IN THIS SCENARIO

web/src/lib/lq-ai/components/ChatPanel.svelte api/app/api/chats.py api/app/audit.py api/app/api/skills.py

api/app/clients/gateway.py gateway/app/tier_floor.py gateway/app/router.py gateway/app/routing_log.py

[Open full-screen ↗](#) [Source: api/app/api/chats.py](#)

3. The tier system: when the Gateway says no

The Gateway enforces five data-sensitivity tiers (Tier 1 = local / air-gapped, most secure; Tier 5 = consumer, least secure). When a request arrives, the Gateway compares the requested provider's tier against the matter's configured floor. A floor of Tier N means "require Tier N or stronger" — if the provider's tier is weaker (higher-numbered) than the floor, the request is refused with a structured error, not a generic 500. This playground lets you set a matter context and a model alias and see whether the request would pass or be refused — the same logic the Gateway runs at

gateway/app/tier_floor.py.

LQ.AI — Tier System & Refusal Explorer

See how the Inference Gateway enforces privilege-tier floors, routes requests, and handles overrides.

↩️

Learn

View

source ↗

PRESET SCENARIOS

Privileged matter, smart model

Privileged matter, fast model

Privileged matter, local model

Standard matter, smart model

Privileged + admin override

Member sees refusal, no override

MATTER CONTEXT

Standard matter

No tier floor — any configured model alias is reachable.

Privileged matter

Tier floor: Tier 2 — requires Tier 2 or stronger (Tier 1 local allowed; Tier 3+ refused).

Confidential matter

Tier floor: Tier 1 (future — local / air-gapped only).

REQUESTED MODEL ALIAS

smart

Tier 4 · Anthropic Claude Sonnet

fast

Tier 3 · fast cloud (e.g. GPT-4o-mini)

Tier 1

Local / air-gapped — most secure (Ollama)

Tier 2

Customer-hosted cloud (operator's VPC / Bedrock / Vertex)

Tier 3

Enterprise cloud with ZDR / no-training commitment

Tier 4

Standard commercial cloud (default; no ZDR)

Tier 5

Consumer / free tier — least secure

⊙ REFUSED

Request refused at tier-floor check

The Gateway enforces this floor before any provider call is made.

REQUESTED TIER

Tier 4

ENFORCED FLOOR

Tier 2 or stronger

MATTER CONTEXT

privileged

USER ROLE

admin

Matter privilege floor is Tier 2 or stronger. The requested alias "smart" resolves to Tier 4, which is weaker than the floor (Tier 4 > Tier 2). The Gateway refuses before forwarding the request to any provider.

REQUEST FLOW

1. Request arrives

→

2. Tier-floor check

→

3. Refuse at Gateway

→

4. Propagate

REFUSAL BUBBLE (WEB/SRC/LIB/LQ-AI/COMPONENTS/REFUSALMESSAGEBUBBLE.SVELTE)

Verify in source

Description

Copy

Key source paths for the current scenario (privileged matter, smart model, admin role).

gateway/app/tier_floor.py

— Tier floor check + matrix evaluation

gateway/app/router.py

— Provider selection and routing

gateway/app/routing_log.py

— Routing log writer

web/src/lib/lq-ai/components/RefusalMessageBubble.svelte

— Refusal UI bubble

web/src/lib/lq-ai/components/TierFloorOverrideModal.svelte

— Override modal component

api/app/api/inference_override.py

— Override endpoint endpoint.svelte

[Open full-screen ↗](#)Source: [gateway/app/tier_floor.py](#)

4. What the model actually sees: Skill Composition

When a skill is invoked, the final prompt the model receives is an assembly of layers: the skill's system prompt, any input variables resolved against the user's message, reference file content, KB chunks retrieved for this specific message, and the conversation history. This playground lets you toggle each layer on or off and watch the assembled context update in real time. The assembly logic lives at [api/app/pipeline/](#).

LQ.AI — Skill Composition Explorer

See exactly what system prompt the model receives when you attach skills to a chat.

[↩️ Learn](#)

[↗️ View source](#)

ATTACH BUILT-IN SKILLS

Select all

Clear all

☐ **action-items-from-client-alert**

Extract time-sensitive action items and deadlines from client alerts, regulatory bulletins, or law firm memos.

☐ **comms-improver**

Rewrite legal-jargon text in plain language for a specified non-legal audience (execs, sales, customers, engineering).

☐ **contract-qa**

Answer specific questions about a loaded contract — clause meaning, unusual provisions, scenario analysis — with verbatim citations.

☐ **dpa-checklist-review**

Score a DPA, HIPAA BAA, or data processing addendum against GDPR Art. 28, US state privacy laws, and commercial benchmarks.

☐ **enhance-prompt**

Expand a short or vague prompt into a structured legal prompt with role, jurisdiction, task, constraints, and output format made explicit.

☐ **gpa-review-commercial-substance**

Check one or more skills on the left to see the assembled system prompt.

Verify in source

Description

Copy

Source paths for every component currently visible in the assembled prompt. Click to open on GitHub.

Assembly logic

- [gateway/app/skills/assembler.py](#)
- [api/app/api/skills.py](#) (built-in skill resolution)
- [api/app/api/user_skills.py](#) (user skill retrieval)

Skill format spec

[format-specification.html](#)

[Open full-screen ↗](#) [Source: api/app/pipeline/](#)

5. Verifying what the model said: Citation Engine cascade

Every "<quote>" (Source: [N]) the model emits runs through a four-stage verification cascade: exact match → tolerant match → paraphrase judge → optional ensemble. The first stage to verify wins; failures cascade. A citation that misses every stage is not persisted — its absence is the "unverified" signal the M2-C2 UI consumes. This playground lets you pick or craft a (source, quote) pair and watch which stage verifies, what the persisted verification_method would be, and how the chat surface would render it.

Citation Engine — 4-stage cascade

Pick a candidate. Watch which stage verifies. The first stage to succeed wins; failures cascade.

source: verification.py

docs

CANDIDATE CITATIONS

Each preset picks a (source, quote) pair. Watch how far the cascade has to go before something verifies — or doesn't.

Exact verbatim quote

Stage 1 wins. Most M2 citations land here in practice.

Whitespace + smart-quotes drift

Stage 1 fails byte-comparison; Stage 2 rapidfuzz catches it.

Legitimate paraphrase

Quotes the meaning, not the verbatim text. Stage 3 judge verifies.

Partial paraphrase

Source supports part of the claim. Stage 3 returns partial.

Fabricated quote

Model invented text not present in the source. Every stage fails; renders as unverified.

EDIT THE SOURCE TEXT

The verifier reads from documents.normalized_content.

The Recipient shall maintain the Confidential Information in strict confidence and shall not disclose such Confidential Information to any third party without the prior written consent of the Discloser.

EDIT THE MODEL'S QUOTE

The string the model emitted as "<quote>" (Source: [N]).

The Recipient shall maintain the Confident:

STAGE 4 — ENSEMBLE

☒ Enable ensemble verification

Replaces Stage 3 single-judge with parallel dispatch across 3 judge models. Stage 1 + 2 still run first.

Activation in production is the OR of: skill frontmatter, project flag, gateway default. Pre-flight cost-budget check can fall back to Stage 3 (see [cost.py](#)).

1

exact_match

Byte-for-byte string match against documents.normalized_content[offset_start:offset_end].

Free (pure Python)

source.index0f(quote) != -1 → true

verified

2

tolerant_match

Normalize whitespace, smart quotes, OCR confusions; rapidfuzz.fuzz.ratio ≥ 95.

Free (pure Python)

skipped

3

paraphrase_judge

LLM judge call through the gateway. Returns yes/partial/no + high/medium/low confidence.

1 judge call per citation (~\$0.001–0.04 depending on model)

skipped

4

ensemble_*

Parallel dispatch across N judge models with strict (unanimous) or majority aggregation.

N judge calls per citation (pre-flight budget check)

skipped

PERSISTED ROW · WHAT MESSAGE_CITATIONS STORES

verification_method

exact_match

confidence

1.00

partial

false

tier_envelope

(single-stage)

verified

UI state: verified (exact/tolerant)

Citation rendered as *The Recipient shall maintain the Confidential Information in strict confidence* — green check + green underline. The model quoted the source faithfully (Stage 1).

[Open full-screen ↗](#) [Source: api/app/citation/verification.py](#)

6. Confidentiality: Anonymization Layer pre/post

The gateway's Anonymization Layer (M2-B3) is pre/post middleware that pseudonymizes detected entities (PERSON, ORGANIZATION, EMAIL_ADDRESS, PHONE_NUMBER, LOCATION, US_BANK_NUMBER + custom CASE_NUMBER and MATTER_NUMBER) before requests leave for the model provider, and rehydrates the pseudonyms on the response. The per-request PseudonymMapper lives in process memory only — never persisted, never logged, dropped on function exit. Privileged-project chats skip the layer entirely; retrieval-context system messages skip via `lq_ai_skip_anonymization` so source quotes reach the model intact for citation grounding. This playground walks the full pipeline with toggles for both skip behaviors.

Honest validation posture: the custom recognizers and middleware integration are tested; Presidio default-recognizer recall/precision on legal-document corpus specifically is empirically unmeasured — docs/security/anonymization.md § "What's validated vs unvalidated" for the risk framing and

route-to-Tier-1 guidance.

Anonymization Layer — pre/post middleware

Watch the gateway pseudonymize before the provider sees it, then rehydrate before you do.

source: `middleware.py` docs

SAMPLE MESSAGES

Each preset shows a different entity mix. The Presidio default set + custom recognizers run on every user message.

NDA review — names + org
Standard PERSON / ORGANIZATION recognizers fire.

Matter intake — case + matter numbers
Custom CaseNumberRecognizer + MatterNumberRecognizer fire.

Contact info — email + phone
EMAIL_ADDRESS + PHONE_NUMBER recognizers fire.

Mixed entities (everything)
Multiple recognizers fire on a single message.

EDIT THE USER MESSAGE

Try adding names, organizations, emails, phone numbers, case numbers, matter numbers. The middleware detects them and substitutes.

Acme Corporation is asking John Smith and Jane Doe to sign an NDA before reviewing the proposal. Can you check the confidentiality definition?

PIPELINE MODE

☐ **Privileged project (skip anonymization)**
Per **Decision A**: rewriting privileged work product risks corrupting it. The chat skips the middleware entirely; tier-floor enforcement bumps routing instead.

☒ **Include retrieval-context system message**
Per **M2-D2**: source documents reach the model intact via `lq_ai_skip_anonymization=true` so citations ground against real text.

1 • USER (WEB CLIENT)

What the user typed

Acme Corporation is asking John Smith and Jane Doe to sign an NDA before reviewing the proposal. Can you check the confidentiality definition?

3 entities detected by Presidio + custom recognizers.

↓ POST /v1/chat/completions ↓

RAG • SYSTEM MESSAGE (RETRIEVED CONTEXT)

api/ marks with `lq_ai_skip_anonymization=true`

[Retrieved context from KB] "The Recipient shall maintain the Confidential Information in strict confidence." (Source: NDA-2024-acme.pdf, p.2)

M2-D2: source documents bypass pseudonymization so the model sees intact source quotes for citation grounding. The skip flag is per-message; chat content above still pseudonymizes.

2 • INFERENCE GATEWAY • PRE_ANONYMIZE_REQUEST

Middleware substitutes detected spans with stable pseudonyms

ORGANIZATION_0001 is asking PERSON_0001 and PERSON_0002 to sign an NDA before reviewing the proposal. Can you check the confidentiality definition?

Per-request PseudonymMapper in process memory. Never persisted; never logged; dropped on function exit.

↓ provider call (Anthropic / OpenAI / Azure / Ollama) ↓

3 • MODEL PROVIDER

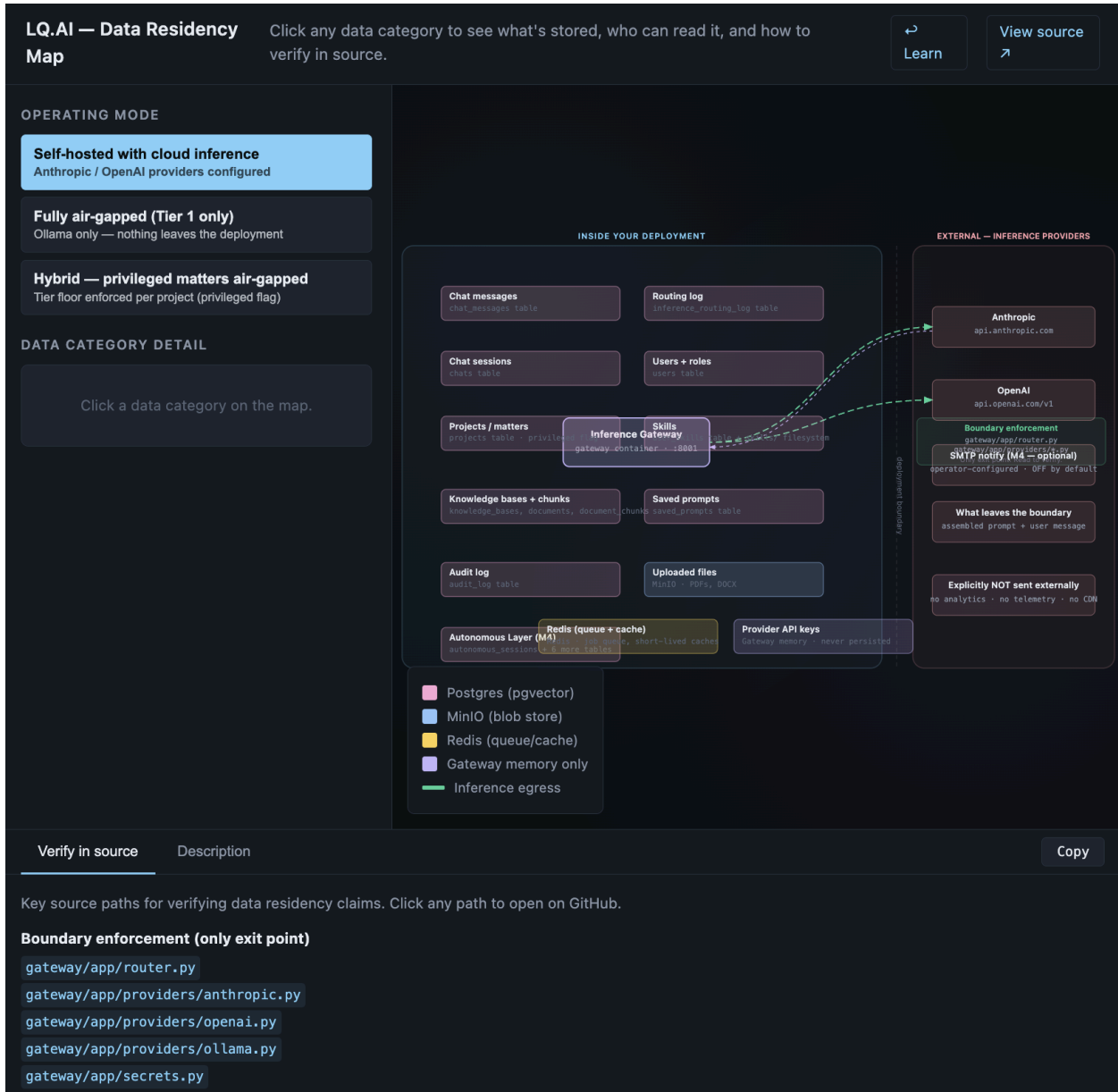
What leaves the deployment (pseudonymized)

ORGANIZATION_0001 is asking PERSON_0001 and PERSON_0002 to sign an NDA before reviewing the proposal. Can you check the confidentiality definition?

[Open full-screen ↗](#)Source: [gateway/app/anonymization/](#)

7. Where your data lives: Data Residency

LQ-AI is self-hosted. By default, all conversation data, knowledge base content, and skill definitions stay within the operator's deployment — they never touch LegalQuants infrastructure. The only outbound path is the inference call from the Gateway to the chosen provider, and only when the matter's tier floor permits it. This map shows every data store, every outbound boundary, and which tiers cross each boundary. The Anonymization Layer middleware (M2-shipped — see playground 6 above) is one of the controls that crosses this surface; this map shows where its pre/post substitution sits relative to every storage and egress point in the system.



8. Reviewing a contract: the Playbook execution cascade

A Playbook codifies an organization's standard positions on common contract issues. Applying one runs a four-node LangGraph cascade — retrieve → classify → redline → compile — that walks each position, extracts the matching clause, classifies it against the standard, and drafts a redline where it deviates. This playground steps through that cascade position-by-position against synthetic NDAs. The per-position references are the verbatim matched clause text (lexical FTS), not the M2 Citation

Engine verification cascade — that integration is deferred.

Playbook executor — 4-node cascade

Pick an NDA. Step the LangGraph cascade — retrieve → classify → redline → compile — position by position.

source: executor.py

docs

TARGET CONTRACT

A synthetic mutual NDA. The executor runs the built-in **NDA — Mutual** playbook (8 positions) against it.

Acme ↔ Beta — compliant, market-standard

THE CASCADE

Four LangGraph nodes run sequentially. Click a node or step forward to advance.

1 retrieve
Postgres FTS over chunks (lexical, top-4)

2 classify
1 structured-JSON LLM call / position

3 redline
1 LLM call / deviates verdict only

4 compile
assemble results JSONB + summary

Reset

Step ▶

Run all

PRESETS

Snap the document + cascade position to an interesting case.

Compliant NDA
Acme ↔ Beta, fully compiled. Mostly matches standard / fallback.

Deviating term (no survival)
Cypress ↔ Delta at classify — watch survival flip to 'deviates'.

See the drafted redline
Cypress ↔ Delta after redline — the survival edit is drafted.

Missing carveout
Gamma ↔ Helix, compiled — two 'missing' verdicts from empty FTS.

Just retrieved
Acme ↔ Beta at the retrieve node — chunks only, no verdicts yet.

EXECUTION IN PROGRESS

Playbook NDA — Mutual · 8 positions · target Acme ↔ Beta — compliant, market-standard · status **running**

— STANDARD

— FALLBACK

— DEVIATES

— MISSING

#0 Definition of Confidential Information

CRITICAL pending

1 · retrieve FTS OVER DETECTION_KEYWORDS → TOP-4 CHUNKS

[chunk 0 · chk_3f0a1]
1. Definitions. "Confidential Information" means any non-public information of either Party furnished to or learned by the other Party, whether in oral, written, electronic, or visual form, that is marked or identified as confidential, or that by its nature would reasonably be understood to be confidential, including business plans, financial information, and source code.

[chunk 1 · chk_3f0a2]
Confidential Information includes information disclosed under circumstances reasonably indicating its confidentiality.

#1 Permitted Disclosures

HIGH pending

1 · retrieve FTS OVER DETECTION_KEYWORDS → TOP-4 CHUNKS

[chunk 0 · chk_3f0b1]
4. Permitted Disclosures. The Receiving Party may disclose Confidential Information to its employees, officers, and professional advisors who have a need to know for the Permitted Purpose and who are bound by confidentiality obligations no less protective than those in this Agreement, and as required by law with prompt prior written notice where legally permitted.

[Open full-screen](#) / [Source: api/app/playbooks/nodes.py](#)

9. Comparing many contracts: the Tabular Review grid

Tabular Review extracts the same set of questions across a set of documents and lays the answers out in a grid — one row per document, one column per question. Each cell is grounded in the chunks the model cited; click a cell to open its citation drawer. This playground renders a small grid of synthetic NDAs; the citation drawer shows the same fields the real surface does. Per-cell citations are display-only chunk references today (a synthetic citation id), not Citation-Engine-resolved provenance.

Tabular Review — grid + cell citation drawer

N documents × M columns. Each cell is one extraction. Click a cell to open its grounding drawer.

docs

PRESETS

Each preset reshapes the grid — different document sets, columns, and confidence profiles.

Contract snapshot — 3 NDAs

3 rows × 3 columns. The default tabular comparison surface.

Full corpus — 5 NDAs

All five sample NDAs × 3 columns. Term spans 1–5 years.

Single column — Term only

One column across 5 docs. The 'compare these on one point' minimal grid.

Failed cell — governing law

Demonstrates the failed-cell state on one cell of the 3-NDA grid.

COLUMNS

Toggle which columns render. Each column is one per-row query dispatched as a structured-output LLM call.

☒ Term

What is the term or duration of the confidentiality obligations?

☒ Confidential Info Definition

How does the agreement define Confidential Information?

☒ Governing Law

What is the governing law of the agreement?

DOCUMENT SET

Rows = documents, in selection order (not sorted by name). Synthetic mutual NDAs from the sample corpus.

☒ 3 NDAs (Acme, Cypress, Echo)

3 documents

☐ 5 NDAs (full corpus)

5 documents

Tabular Review runs the same per-position extraction over many documents at once. Retrieval is lexical full-text search (FTS) over each document's chunks, top-K = 4; one structured-output LLM call per cell returns {value, cited_chunk_indices, confidence, justification}. Click any cell to open the per-cell citation drawer.

DOCUMENT	TERM	CONFIDENTIAL INFO DEFINITION	GOVERNING LAW
nda-1-acme-beta.pdf Acme ↔ Beta	3 years from the Effective Date. HIGH	Broad — oral and written information, marked or not. HIGH	State of Delaware
nda-2-cypress-delta.pdf Cypress ↔ Delta	2 years from the Effective Date. HIGH	Narrow — written and marked only; oral confirmed within 30 days. MED	State of Delaware
nda-3-echo-foxtrot.pdf Echo ↔ Foxtrot	5 years from the Effective Date. HIGH	Broad, with explicit enumerated categories. MED	Delaware (governing law not separately stated venue clause)

● high — stated explicitly

● medium — implied, not verbatim

● low — weakly supported; flag for review

● failed — extraction errored (not "unverified")

schema_version m3-c2-v1

total_cells 9

failed_cells 0

cost_actual_usd 0 (DE-310)

source: [api/app/tabular/nodes.py](#) [web/src/lib/lq-ai/components/TabularGrid.svelte](#)

[Open full-screen ↗](#)Source: [api/app/tabular/nodes.py](#)

10. Into the editor: the Word add-in install + auth flow

The Word add-in is an Office.js task pane installed against the operator's own deployment. This playground walks the four-stage flow: admin generates a per-deployment manifest, the operator sideloads the unsigned manifest via the Microsoft 365 Admin Center (which warns about the unsigned add-in — expected at v0.3.0), the task pane completes OAuth against the deployment, and the version handshake confirms compatibility. M3 shipped the plumbing only — the in-pane feature surface (chat, skills, playbooks) is deferred (DE-287; M4 closed without it — community-friendly),

and the signed distribution package is community-led.

Word Add-In — install

+ auth flow

Sideload an unsigned manifest, OAuth against your deployment, and check the M3-B8 version handshake. Step through it.

source:

word_addin.py

docs

STEP THROUGH THE FLOW

Four stages: generate manifest → sideload (unsigned) → OAuth → version handshake. The simulated task pane on the right reflects the current step.

< Back

Next >

Step 1 of 4 — GET /admin/word-addin/manifest

PRESETS

Each preset sets the step + version-handshake state to a representative scenario.

Fresh install

Walk steps 1→4 with a compatible add-in. The happy path.

Compatible (signed-in)

Jump to step 4, version compatible. Authenticated tab strip + deep-link placeholders.

Version drift (add-in outdated)

Step 4 with installed 0.2.4 < min 0.3.0 → blocking Update-needed overlay.

Deployment outdated (add-in too new)

Step 4 with installed 0.4.0 > max 0.3.99 → 'update the deployment' overlay.

Handshake unreachable

Step 4, version endpoint fails → soft non-blocking banner; the pane still renders.

VERSION-HANDSHAKE STATE

What GET /api/v1/word-addin/version resolves to once the task pane runs classifyVersion() on mount. Affects step 4.

compatible — within range

MANIFEST TOKENS

Substituted into the XML by render_manifest() at generate time.

☐ Custom brand / provider

Set display_name=Acme Legal & provider_name=Acme Corp query params (default)

What actually works at v0.3.0: install (unsigned manifest) · OAuth · version handshake — plumbing only. The in-pane feature surface (chat, skills, playbooks, the Inference Tier badge) is **not built** — deferred to M4 (DE-287); each authenticated tab is a deep-link card to the web app. A **signed manifest** + enterprise distribution package is community-led (DE-295). This playground depicts those as placeholders, never as working features.

1

GET /admin/word-addin/manifest

Admin opens /lq-ai/admin/word-addin → Generate manifest. The backend templates the deployment URL + a fresh uuid4 into the XML and returns application/xml as an attachment.

display_name=LQ.AI · provider_name=LegalQuants · DEPLOYMENT_ORIGIN=https://lq.acme.example · ADDIN_ID=<uuid4> · <Version>0.3.0.0</Version>

running

2

Sideload (UNSIGNED) via M365

Admin uploads the manifest in Microsoft 365 Admin Center → Integrated apps → Upload custom apps → Office Add-in. M365 warns it is unsigned — expected at v0.3.0.

pending

3

OAuth via Office.js Dialog API

Task pane loads in the sandboxed iframe; SignInGate opens displayDialogAsync against the deployment's own login form (its existing JWT issuer — no MSAL). messageParent posts the session back.

pending

4

GET /word-addin/version handshake

On mount (before sign-in) the pane calls the unauthenticated version endpoint and runs classifyVersion(). Out-of-range → blocking Update-needed overlay; in-range → normal layout.

pending

SIMULATED WORD TASK PANE

File Home Insert Review

No add-in installed yet

Admin generates the manifest from https://lq.acme.example/lq-ai/admin/word-

[Open full-screen ↗](#)Source: [api/app/api/word_addin.py](#)

11. Seeing it all at once: the observability trace

Every chat-send is one OpenTelemetry trace spanning `api` → `gateway` → `provider`, with domain spans for citation verification, anonymization, skill dispatch, and inference carrying counts and types only — never raw entity values, prompt text, or response content. This playground lets you toggle the citation path, anonymization state, workflow type, and provider tier to see how the span tree changes, and shows which span attribute answers each of the five questions an operator is most likely to ask in production.

OTel trace — chat-send end to end

Every chat-send is one distributed trace. Explore the span hierarchy, domain attributes, and operator questions.

[docs/observability.md](#)

[observability_helpers.py](#)

CITATION PATH
Determines the `citation.verify` subtree shape.

Exact-match hit (fast)
Stage 1 wins — short-circuit event; no tolerant/judge spans emitted.

Ensemble (3 judges)
Stages 1+2 miss; cascade goes to ensemble. 3 parallel judge spans.

ANONYMIZATION
☒ **Anonymization active**
Shows `anonymization.pre` / `.post` spans + `entity_count` attribute. Uncheck to simulate privileged project (`skip_reason = "privileged"`).

WORKFLOW

Chat
Plain chat-send. One skill, one inference call.

Playbook (30 positions)
`playbook.execute` spans each position sequentially.

Tabular (5 docs × 4 cols)
`tabular.execute` spans each cell in a 5 × 4 grid.

PROVIDER / TIER
Sets attributes on the `inference.dispatch` span.

Anthropic · claude-opus-4-7 · Tier 5 (cloud) ▾

SPAN TREE — CLICK ANY SPAN TO EXPAND ATTRIBUTES

POST /api/v1/chats/{chat_id}/messages (api — SERVER) 890ms

└ **skill.execute** (nda-review) INTERNAL 885ms

└ **POST /v1/chat/completions** (api → gateway — CLIENT) 3.88s

└ **inference.dispatch** (gateway) INTERNAL 3.87s

└ **anonymization.pre** (gateway) INTERNAL 18ms

POST https://api.anthropic.com/v1/messages (provider — CLIENT, httpx auto-instrumented)

└ **anonymization.post** (gateway) INTERNAL 7ms

└ **citation.verify** (api) INTERNAL 2ms

└ **citation.stage.exact_match** INTERNAL 1ms

5 OPERATOR QUESTIONS — WHICH SPAN ANSWERS EACH ONE

Why was this slow?
Compare the duration bars. The longest bar is your bottleneck. For cloud providers, `inference.dispatch` dominates. For playbooks, `playbook.execute` stacks 30 sequential positions.

How much did this inference call cost?
`inference.dispatch` → attribute `inference.cost_usd = 0.0613` (provider = anthropic, model = claude-opus-4-7, tier = 5).

Did anonymization run? How many entities?
`anonymization.pre` → `anonymization.entity_count = 3`, `entity_types = ["PERSON", "ORGANIZATION", "MATTER_NUMBER"]`.

[Open full-screen ↗](#) Source: [docs/observability.md](#)

12. Autonomy you can audit: the Autonomous flow

The Autonomous Layer (M4, shipped) runs a single agent on your behalf — on a schedule, or when documents arrive — without you watching each step. Because no human approves each action, the agent runs through declared phases behind one brake-checked chokepoint, and every run produces an auditable receipt. Step through a session below and trip each brake yourself.

Autonomous flow — a single agent, fully audited

Step a single-agent session through its phases and watch the brakes fire

M4 — SHIPPED (OPT-IN, OFF BY DEFAULT)

How aligned flows are built ↗

DRIVE THE SESSION

Each step is one tool call through the chokepoint. Auto-run plays the session to its terminal state.

Step ▶

Auto-run ▶▶

Reset ↺

COST CAP (R4)

Session field max_cost_usd — the gateway.yaml default is \$5. Lowering it mid-run trips Scenario A.

max_cost_usd

\$5.00

BRAKES

Halt (the red button)

Sets halt_state = halt_requested; honored at the next tool boundary (Scenario B).

Force ungranted call

Attempt a tool not granted in the current phase (Scenario C). The stripped chip flashes red.

Idle past idle_halt_minutes (5)

Simulate inactivity: the watchdog ticks running → paused → halted (Scenario D).

BRAKE SCENARIOS (A-D)

Each maps one-to-one to an M4-A3 acceptance test. Trigger one to see its outcome + span attribute + audit row.

A • R4 cost cap

Projected cost > max_cost_usd → cost_cap_reached, partial result kept.

B • R5 external halt

Halt mid-run → next call reads halt_requested → external_halt.

C • R6 phase-grant

Autonomous tool call during ethics review — tool not granted

ONE AUTONOMOUS_SESSION — FIVE PHASES, LEFT TO RIGHT

intake

retrieve_chunks

run_skill

run_playbook

propose_memory

propose_precedent

emit_finding

notify

→

analysis

retrieve_chunks

run_skill

run_playbook

propose_precedent

propose_memory

emit_finding

notify

→

drafting

run_skill

propose_memory

propose_precedent

emit_finding

retrieve_chunks

run_playbook

notify

→

ethics_review

emit_finding

retrieve_chunks

run_skill

run_playbook

propose_memory

propose_precedent

notify

THE GUARDED_TOOL_CALL CHOKEPOINT — FOUR ORDERED CHECKS

R5 → R6 → R4 → open span + audit + run + record

R5 read halt_state (before the call)

R6 phase grants this intent?

R4 projected cost ≤ max_cost_usd?

→ open OTel span + write audit row → run tool → record cost + outcome

There is exactly one chokepoint.

Every tool call goes through here; a new tool gets the brakes for free and cannot route around them. The order is fixed: R5 (halt) → R6 (phase grant) → R4 (cost) — all before the tool runs.

AUTONOMOUS.TOOL_CALL SPAN + AUDIT ROW — THIS STEP

OTEL SPAN — AUTONOMOUS.TOOL_CALL

AUDIT ROW — AUDIT_LOG

Step the session to emit a span.

Step the session to write an audit row.

THE "COUNTS ONLY" GUARANTEE — WHAT APPEARS VS. WHAT NEVER APPEARS

APPEARS IN SPAN ATTRS + AUDIT

NEVER APPEARS ANYWHERE

[Open full-screen ↗](#)Source: [agentic-flow-alignment-guide.md](#)

13. The four autonomous primitives: watches, schedules, memory, precedent

An autonomous session does not run in isolation — it is wired to four primitives that decide when it runs and what it carries across runs. Watches trigger a session when matching documents arrive; schedules trigger it on a cron-like cadence; memory persists what a session learned so later runs build on it; and the precedent lifecycle promotes vetted work product into reusable precedent. This playground steps through each primitive and shows how it feeds the session you saw in playground

12. The Autonomous Layer shipped in M4.

LQ.AI LEARN · AUTONOMOUS

Autonomous primitives: watches · schedules · memory · precedent

The four building blocks of the autonomous layer. Each has its own trigger → session → outcome lifecycle. Pick a primitive, then click a step to see the real endpoints, tables, and state transitions behind it. (For what happens *inside* a single session — its phases and brakes — see the autonomous-flow playground.)

The whole layer is **per-user opt-in, off by default** (`User.autonomous_enabled`). Every state transition writes an audit row carrying counts, types, IDs, and enum values only — never raw document text or proposed values.

01
Watches
`autonomous_watches`

02
Schedules
`autonomous_schedules`

03
Per-user memory
`autonomous_memory`

04
Precedent board
`precedent_entries`

Watches SHIPPED · M4

A **watch** binds an autonomous session to a knowledge base. Attaching a document to that KB triggers a session for every active watch on the KB. Each trigger is bounded by the watch's per-trigger cost cap.

`autonomous_watches (0039)``max_cost_usd col (0045)`

LIFECYCLE — CLICK A STEP

01
Create watch
`setup`

02
Doc attached
`trigger`

03
Session runs
`session`

Create a watch on a KB

The user creates a watch bound to a knowledge base. `max_cost_usd` (added in migration 0045) sets the per-trigger budget cap. Watches are listed and deleted from the same resource.

`GET /autonomous/watches``POST /autonomous/watches``DELETE /autonomous/watches/{watch_id} (soft-delete)`

[Open full-screen ↗](#)Source: `api/app/api/autonomous.py`

14. Finding the right chunks: knowledge-base hybrid retrieval

When a message needs grounding context, LQ-AI does not rely on vector search alone. It runs hybrid retrieval: a lexical full-text-search pass (Postgres FTS) and a vector cosine-similarity pass run in parallel, and their results are fused into a single ranked set so that both exact-term matches and semantically-related passages surface. This playground lets you issue a query against synthetic KB chunks and watch the lexical scores, the vector scores, and the fused ranking that the engine ultimately uses. Knowledge-base retrieval shipped in M1.

Knowledge bases: hybrid retrieval (lexical + vector)

How a query becomes grounded context for chat, skills, and citation verification.

[↩ Learn](#) [source: retrieval.py ↗](#)

SAMPLE QUERY

Each query fans into two retrieval arms over the same KB chunks, then the arms are merged. The lexical arm rewards literal term overlap; the vector arm rewards meaning.

"who indemnifies the buyer"
literal vs semantic term split

"how long does confidentiality last"
duration / survival phrasing

MIX WEIGHT **HYBRID_ALPHA**

Per-KB setting (default 0.5), overridable per query. 0 = vector-only, 1 = lexical/FTS-only. The fused score is $(1 - \alpha) \cdot \text{vector} + \alpha \cdot \text{fts}$.

alpha **0.50**

0 · vector 1 · lexical

RESULT COUNT **TOP_K**

Each arm pulls `top_k × 4` candidates (`CANDIDATE_OVERSHOOT`) before merge. Default 10, max 50.

4 final chunks returned

FTS-ONLY FALLBACK

If the query embedding can't be produced (gateway down) or $\alpha = 1$, the vector arm is skipped and ranking degrades to FTS-only — retrieval still works.

1 · INGESTION — DOCUMENT → CHUNKS → TSVECTOR + EMBEDDING → READY

pending
queued

parsing
PDF / DOCX / txt

chunking
2000c · 200 ovl

embedding
→ vector(1536)

indexing
content_tsv (gen)

ready
searchable

Chunking is a **character-precise sliding window** (`DEFAULT_TARGET_CHARS = 2000`, `DEFAULT_OVERLAP_CHARS = 200`) that snaps boundaries back to the nearest sentence/paragraph break. The invariant `content == normalized_content[char_offset_start:char_offset_end]` is what the citation engine later verifies against. Each `document_chunks` row carries a **generated** `content_tsv` tsvector (for the lexical arm) and an `embedding vector(1536)` (for the vector arm). A file is only searchable once its `files.ingestion_status = 'ready'`.

2 · THE QUERY FANS INTO TWO ARMS

QUERY "who indemnifies the buyer" **embedded → 1536-dim vector**

Both arms scan `document_chunks` joined to `documents` → `files` → `knowledge_base_files`, filtered to `files.ingestion_status = 'ready'` and not soft-deleted, in this KB. Showing the top candidates per arm.

LEXICAL **Postgres full-text search**

Stemmed, operator-safe; GIN-indexed tsvector.

```
rank = ts_rank_cd(
  dc.content_tsv,
  plainto_tsquery('english', :q)
)
```

VECTOR **pgvector cosine**

Query embedding via the Inference Gateway.

```
score = 1.0 - (
  dc.embedding <=> CAST(:q AS vector)
)
```

HOW THIS WORKS IN THE CODEBASE

Hybrid retrieval is M1 (shipped). The lexical arm is Postgres FTS — `ts_rank_cd` over a generated `content_tsv` tsvector, not a literal BM25 library. The vector arm is pgvector cosine distance (`<=>`). Fusion is a **min-max-normalized weighted linear combination** via `hybrid_alpha` — not Reciprocal Rank Fusion. The pgvector ANN index (ivfflat/hnsw) is deferred, so the vector arm is a sequential cosine scan today. Mechanics, operators, and constants below are read from source; the sample chunks and scores are illustrative.

```
api/app/knowledge/retrieval.py — hybrid_search; _FTS_SQL ( ts_rank_cd / plainto_tsquery('english', _) ); _VECTOR_SQL ( 1.0 - (embedding <=> _) ); _min_max_normalize + (1-alpha)·v + alpha·f; CANDIDATE_OVERSHOOT = 4
api/app/knowledge/embed.py — request_embedding_vector via the gateway; alias "embedding" → text-embedding-3-small, EMBEDDING_DIMENSION = 1536
```

[Open full-screen ↗](#) [Source: api/app/knowledge/retrieval.py](#)

15. The matter's context: projects, org profile, and tier floors

Every request runs inside a matter (a project), and the matter carries the context that shapes it: the organization profile (the house voice and standard positions), the matter's attachments and knowledge bases, and its privilege flag and tier floor. This playground lets you configure a matter and watch how each of those settings flows into the assembled request — including how a privileged matter or a stricter tier floor narrows which providers the Gateway will permit. Projects, organization profiles, and tier floors shipped in M1.

LQ.AI — Matters, org profile & privilege tiers

How a matter's context shapes a request — and how its privilege posture sets a tier floor the gateway enforces.

[Learn](#)[View source ↗](#)

MATTER (PROJECT) M1

Standard matter
privileged=false · minimum_inference_tier NULL — no floor; any tier may serve.

Privileged matter
privileged=true · minimum_inference_tier=2 — Tier 2 or stronger required.

Privileged, local-only
privileged=true · minimum_inference_tier=1 — local / air-gapped only.

ATTACHED TO THE MATTER M1

☒ **Attached files**
Documents uploaded for this engagement.

☒ **Attached skills**
Skills selected for this matter's work (each may declare its own tier floor).

☐ **Knowledge bases**
project_knowledge_bases junction (migration 0021).

ORGANIZATION PROFILE M1

☒ **Org profile configured**
Singleton skill (is_organization_profile=true). Voice + "what good looks like" injected ambiently.

CANDIDATE PROVIDER TIER

[How it works](#)[Verify in source](#)

A **matter** (a Project, PRD §3.11, M1) is a workspace whose context shapes every request made inside it. Three things compose into a request:

- **The organization profile** (PRD §3.12, M1) — a deployment singleton implemented as a skill with `is_organization_profile=true` frontmatter. When configured, its voice and standards inject as ambient context into every chat and skill execution.
- **Attached files, skills, and knowledge bases** — per-matter resources. Skills compose into the system prompt; knowledge bases attach via the `project_knowledge_bases` junction (migration 0021).
- **The privilege posture** — the project's `privileged` flag and `minimum_inference_tier` column. A privileged matter sets a tier floor.

The privilege floor is the security-boundary piece (PRD §3.13, M1). The backend forwards the project floor to the gateway as `lq_ai_project_minimum_inference_tier`. The gateway resolves the effective floor with `resolve_tier_floor()` — `min()` across the request override, the

A · WHAT CONTEXT COMPOSES INTO THE REQUEST

org Organization profile (singleton skill) — org-wide voice, templates, and "what good looks like" injected as ambient context.

matter Matter: `privileged=false · minimum_inference_tier=NULL`

files Attached files — extracted document text for this engagement.

skills Attached skills — each skill body composes into the system prompt; a skill may also declare its own `minimum_inference_tier`.

message User message / document under review.

B · PRIVILEGE POSTURE → TIER FLOOR (FORWARDED TO THE GATEWAY)

T1 local Ollama / air-gapped ALLOWED	T2 customer-hosted cloud ALLOWED	T3 enterprise ZDR / no-training ALLOWED	T4 standard cloud API ALLOWED	T5 consumer / free ALLOWED
--	--	---	---	--

privileged=false, so minimum_inference_tier is NULL (CHECK `chk_projects_privileged_implies_tier` only requires a tier when privileged=true). No project floor is forwarded; the request is never refused on the tier-floor axis.

`resolve_tier_floor()` returns `None`, so `is_refused()` is always false.

C · GATEWAY DECISION FOR THE CANDIDATE PROVIDER

[Open full-screen ↗](#)Source: [api/app/api/projects.py](#)

16. Getting work in: the Slack/Teams intake bridges

The intake bridges let an operator connect a Slack workspace or a Microsoft Teams tenant so that requests can flow into LQ-AI from where the business already works. This playground walks the OAuth install flow and the admin lifecycle (list, soft-delete) for a connected bridge.

Honest partial state: the backend plumbing and admin lifecycle shipped in M3, but the live OAuth install handshake is unverified against real Slack and Teams tenants (DE-312), and the inbound /lq command path is inert — it does not yet dispatch a request (DE-288). Treat this surface as

scaffolding, not a production-ready intake channel.

